Lean 4

Task	Command	Notes
Version manager	`elan`	`elan self update` to update
New project	`lake init` *name* `math`	`math` template adds Mathlib
Add dependency	edit `lakefile.lean`	`require` *pkg* `from git` *url*
Install deps	`lake update`	also: `lake exe cache get` for Mathlib oleans
Build	`lake build`	`lake clean` to rebuild
Run	`lake env lean` *file*`.lean`	
REPL	`lake repl`	
Test	`lake test`	requires test target in lakefile
Format	—	style is by convention
Lint	—	LSP provides diagnostics
LSP	`lean -server`	built-in; the editors start it

Basic Syntax

`def f (x : Nat) : Nat`	Function definition
`let x :=` *expr*	Local binding
`where`	Local definitions
`fun x =>` *expr*	Lambda
`match x with`	Pattern match
`| pat =>` *expr*	Match arm
`if h : p then ... else`	If (with opt. proof)
`for x in xs do`	For loop (monadic)
`do`	Monadic block
`return` *e*	Return in do block

Types & Data

`structure S where`	Record type
`inductive T where`	Sum type / enum
`class C where`	Typeclass
`instance : C T`	Typeclass instance
`abbrev`	Transparent def
`deriving Repr, BEq`	Auto-derive
`(`*a*`, `*b*`)`	Product / tuple
`#[`*a*`, `*b*`]`	Array literal
`[`*a*`, `*b*`]`	List literal

Array Operations

`a[i]!`	Index (panics OOB)
`a[i]?`	Index (returns Option)
`a.push x`	Append element
`a.size`	Length
`a.map f`	Map over array
`a.filter p`	Filter array
`a.foldl f init`	Left fold
`a.toList`	Convert to List
`a.set! i x`	Update at index
`a ++ b`	Concatenate arrays

String & IO

`IO.println s`	Print line
`IO.FS.readFile path`	Read file
`IO.FS.writeFile path s`	Write file
`toString x`	Convert to string
`s.length`	String length
`s.append t` / `s ++ t`	Concatenate
`s.splitOn ","`	Split string
`s.trim`	Trim whitespace

Common Typeclasses

`ToString`	String conversion
`Repr`	Debug representation
`BEq`	Boolean equality
`Ord`	Ordering / comparison
`Hashable`	Hash support
`Inhabited`	Default value
`Add, Mul, ...`	Arithmetic ops
`Monad, Functor`	FP abstractions

Inspecting

`#eval expr`	Evaluate expression
`#check expr`	Show type
`#check @name`	Show with implicits
`#print name`	Print definition
`open Ns in`	Open namespace

Theorem Proving

`theorem n : P := by`	Prove by tactics
`example : P := by`	Anonymous proof
`lemma`	Same as theorem
`sorry`	Placeholder (hole)

Core Tactics

`exact e`	Provide exact term
`apply f`	Apply function/lemma
`intro h`	Introduce hypothesis
`cases h`	Case split
`induction n`	Induction
`constructor`	Split goal (And/Exists)
`rfl`	Reflexivity
`rw [h]`	Rewrite with h
`rw [← h]`	Rewrite backward
`simp`	Simplification
`ring`	Ring equations
`omega`	Linear integer arith
`linarith`	Linear real arith
`norm_num`	Numeric normalization
`contradiction`	Close by contradiction

Tactic Combinators

`have h : P := by`	Intermediate lemma
`suffices h : P by`	Backward reasoning
`obtain ⟨a, b⟩ := h`	Destructure
`calc`	Calculational proof

Mathlib

`import Mathlib.Tactic`	Import tactics
`import Mathlib.Data.Nat`	Specific module
`exact?`	Search for proof term
`apply?`	Search for lemma
`rw?`	Search for rewrite
`simp?`	Show simp lemmas used

Functional Patterns

`List.map f xs`	Apply f to each element
`List.filter p xs`	Keep elements where p
`List.foldl f init xs`	Left fold
`List.foldr f init xs`	Right fold
`List.zip xs ys`	Pair elements
`List.enum xs`	Index + element
`List.any p xs`	Existential
`List.all p xs`	Universal
`List.filterMap f xs`	Map + filter None
`List.bind xs f`	flatMap / concatMap
`xs ++ ys`	Concatenate
`xs |>.map f |>.filter p`	Pipeline syntax
`f $ x`	Application (low prec)
`f <$> x`	Functor map
`f <*> x`	Applicative apply
`x >>= f`	Monadic bind

Higher-Order & Composition

`fun x => f (g x)`	Compose (no operator)
`Function.comp f g`	Composition function
`(f a b ·)`	Partial application
`(· + 1)`	Anonymous function
`@[simp] def ...`	Tag for simp lemma
`@[inline] def ...`	Inline hint
`partial def f`	Non-terminating fn
`termination_by ...`	Termination proof

Monads & Do Notation

`Option`	Maybe monad
`Except E A`	Result / Either
`IO A`	Side effects
`StateM S A`	Stateful computation
`ReaderM R A`	Reader / environment
`do let x ← ma`	Bind in do block
`pure a`	Wrap in monad
`MonadLift`	Lift between monads

Neovim (lean.nvim)

`<Leader>i`	Toggle infoview
`<Leader>p`	Pin infoview to cursor
`<Leader>x`	Unpin infoview
`<Leader>t`	Replace sorry
`K`	Hover info
`gd`	Go to definition
`gr`	References

Lean 4 — Mathlib Reference

Algebraic Structures

`AddGroup`	Additive group
`Group`	Multiplicative group
`CommRing`	Commutative ring
`Field`	Field
`Module R M`	Module over ring R
`Algebra R A`	R-algebra
`LinearMap`	R-linear map
`RingHom`	Ring homomorphism
`Subgroup G`	Subgroup of G
`Ideal R`	Ideal of ring R
`Quotient`	Quotient structure

Number Theory

`Nat.Prime p`	p is prime
`Nat.gcd a b`	Greatest common divisor
`a ∣ b`	a divides b
`a % b`	Remainder
`ZMod n`	Integers mod n
`Int.emod`	Integer mod
`Finset.range n`	{0, . . . , n-1}
$\sum$ `i in s, f i`	Finite sum
$\prod$ `i in s, f i`	Finite product

Sets & Logic

`Set α`	Set type
`x ∈ s`	Membership
`s ∪ t`	Union
`s ∩ t`	Intersection
`s ⊆ t`	Subset
`s \ t`	Set difference
`s.image f`	Image of f
`s.preimage f`	Preimage of f
`Finset α`	Finite set
`∀ x, P x`	Universal
`∃ x, P x`	Existential
`P ∧ Q`	And
`P ∨ Q`	Or
`P → Q`	Implication
`¬ P`	Negation
`P ↔ Q`	Iff

Order & Lattices

`PartialOrder`	≤ partial order
`LinearOrder`	Total order
`Lattice`	Sup + inf
`CompleteLattice`	Arbitrary sup/inf
`sup a b` / `inf a b`	Join / meet
`Filter`	Mathlib filter
`Filter.Tendsto`	Limit via filters

Analysis — Reals

ℝ (`Real`)	Real numbers
ℚ (`Rat`)	Rationals
ℂ (`Complex`)	Complex numbers
`abs x`	Absolute value
`Real.sqrt x`	Square root
`Real.exp x`	Exponential
`Real.log x`	Natural log
`Real.sin x` / `cos x`	Trig functions

Analysis — Limits & Continuity

`Filter.Tendsto f l l'`	f tends to l' along l
`nhds a`	Neighborhood filter
`atTop`	Filter at $+\infty$
`Continuous f`	f is continuous
`HasDerivAt f f' x`	Derivative at x
`Differentiable ℝ f`	f differentiable
$\int$ `x in a..b, f x`	Interval integral

`MeasureTheory.Integrable`
Lebesgue integrable

Topology

`TopologicalSpace`	Topology typeclass
`IsOpen s`	s is open
`IsClosed s`	s is closed
`IsCompact s`	s is compact
`IsConnected s`	s is connected
`MetricSpace`	Metric space
`dist x y`	Distance
`NormedAddCommGroup`	Normed group

Linear Algebra

`Matrix m n α`	m × n matrix
`Matrix.mul A B`	Matrix multiply
`Matrix.det A`	Determinant
`Matrix.transpose A`	Transpose
`LinearMap.ker f`	Kernel
`LinearMap.range f`	Image / range
`Submodule R M`	Submodule
`Basis`	Basis of module
`FiniteDimensional`	Finite dim space

Key Imports

```
Mathlib.Tactic
Mathlib.Data.{Nat,Int,Real,Finset}.Basic
Mathlib.Algebra.Group.Basic
Mathlib.Analysis.SpecificLimits.Basic
Mathlib.Topology.Basic
Mathlib.LinearAlgebra.Matrix.Determinant
```

Proof Search Tactics

`exact?`	Find exact proof term
`apply?`	Find applicable lemma
`rw?`	Find rewrite rule
`simp?`	Show which simp lemmas
`decide`	Decidable propositions
`norm_num`	Numeric computation
`positivity`	Prove positivity
`polyrith`	Polynomial arithmetic
`aesop`	Automated reasoning
`gcongr`	Congruence for ≤
`field_simp`	Clear denominators
`push_neg`	Push negation inward